

MIP - Mainframe Intelligence Platform

Version 1.0

Executive Summary

Mainframe systems continue to power many of the world's largest enterprises, including banking, financial services, insurance, healthcare, telecommunications, government, and retail organizations.

These systems are highly reliable, performant, scalable, and business-critical. However, over decades of evolution, business knowledge has become deeply embedded within source code, batch jobs, database structures, and operational processes.

Organizations face increasing challenges in understanding, maintaining, enhancing, and modernizing these applications due to retiring experts, growing complexity, and the need to deliver new digital capabilities.

The Mainframe Intelligence Platform (MIP) is designed to become the enterprise system of intelligence for legacy applications by continuously discovering, modeling, explaining, and modernizing mainframe ecosystems.

Vision

Enable organizations to understand, evolve, modernize, and transform legacy applications with confidence, speed, and reduced risk.

MIP transforms tribal knowledge into enterprise knowledge.

Problem Statement

Large enterprise legacy applications face the following challenges:

Knowledge Attrition

Critical system knowledge resides with a small number of experts.

Knowledge is hidden inside:

- COBOL
- JCL
- Copybooks

- DB2
- VSAM
- IMS
- CICS
- Scheduler Definitions
- Operational Runbooks

As experienced engineers retire, organizations lose decades of institutional knowledge.

Monolithic Complexity

Applications often contain:

- Thousands of programs
- Thousands of jobs
- Shared copybooks
- Complex batch chains
- Deeply coupled dependencies

Understanding system behavior becomes increasingly difficult.

Slow Change Velocity

Even minor enhancements require extensive analysis to determine:

- Impacted programs
- Impacted tables
- Business rules
- Downstream dependencies

This increases delivery timelines and operational risk.

Modernization Risk

Organizations struggle to answer:

- What should be modernized first?
 - What business capabilities exist?
 - What services can be extracted?
 - What dependencies must be preserved?
 - How can modernization be performed safely?
-

What Is Not The Problem

Mainframes continue to provide:

- Exceptional throughput
- Predictable latency
- Reliability
- Transaction integrity
- Operational stability
- Proven scalability

The challenge is not performance.

The challenge is understanding and evolving the system safely.

Mission

Convert legacy application understanding from a human-dependent activity into a continuously updated intelligence capability.

Target Industries

- Banking
 - Financial Services
 - Insurance
 - Healthcare
 - Telecommunications
 - Government
 - Retail
 - Logistics
 - Manufacturing
-

Primary Users

Developers

Need:

- Program understanding
 - Dependency analysis
 - Impact assessment
-

Application Support Teams

Need:

- Incident analysis
 - Root cause discovery
 - Operational visibility
-

Architects

Need:

- System understanding
 - Domain boundaries
 - Modernization opportunities
-

QA Teams

Need:

- Regression analysis
 - Impacted functionality identification
-

Business Analysts

Need:

- Business flow understanding
 - Capability mapping
-

Modernization Teams

Need:

- Service extraction guidance
 - Migration recommendations
 - Transformation strategies
-

Product Philosophy

Do not start with AI.

Start with facts.

Source Code ↓ Metadata ↓ Knowledge Graph ↓ Reasoning ↓ AI

AI consumes knowledge.

AI does not replace knowledge.

Platform Architecture

Layer 1 - Source Systems

Artifacts:

- COBOL
 - JCL
 - Copybooks
 - DB2
 - VSAM
 - IMS
 - CICS
 - Scheduler Definitions
 - Control-M
 - CA7
 - Documentation
-

Layer 2 - Discovery Layer

Responsibilities:

- Repository discovery
- Artifact classification
- Metadata extraction

Components:

- Repository Scanner
 - COBOL Parser
 - JCL Parser
 - Copybook Parser
 - DB2 Parser
 - VSAM Parser
 - CICS Parser
-

Layer 3 - Metadata Repository

Centralized metadata storage.

Stores:

- Program metadata
 - Job metadata
 - Copybook metadata
 - Table metadata
 - Transaction metadata
 - Dataset metadata
-

Layer 4 - Knowledge Graph

Enterprise relationship model.

Stores:

- Dependencies
 - Data lineage
 - Call hierarchy
 - Business capability mapping
-

Layer 5 - Reasoning Layer

Provides:

- Root discovery
 - Dependency tracing
 - Impact analysis
 - Flow analysis
 - Business rule extraction
-

Layer 6 - AI Copilot

Provides:

- Natural language Q&A
 - Documentation generation
 - Architecture explanations
 - Modernization recommendations
-

Metadata Model

Program

Attributes:

- Program Name
 - Type
 - Source Location
 - Business Domain
 - Last Modified Date
-

Job

Attributes:

- Job Name
 - Scheduler
 - Frequency
 - Owner
-

Copybook

Attributes:

- Copybook Name
 - Shared Structures
 - Field Definitions
-

Table

Attributes:

- Table Name
 - Database
 - Schema
 - Business Domain
-

Dataset

Attributes:

- Dataset Name

- Dataset Type
 - Access Pattern
-

Transaction

Attributes:

- Transaction Name
 - CICS Program
 - Business Function
-

Business Capability

Examples:

- Authorization
 - Payment Processing
 - Statement Generation
 - Collections
 - Fraud Management
 - Customer Service
 - Rewards
-

Business Rule

Examples:

- Decline if balance exceeds limit.
 - Apply late fee after due date.
 - Generate statement monthly.
-

Knowledge Graph Model

Nodes

- Program
- Job
- Table
- Copybook
- Dataset
- Transaction
- Business Capability
- Business Rule
- Service Candidate

Relationships

CALLS

Program → Program

EXECUTES

Job → Program

USES

Program → Copybook

READS

Program → Table

Program → Dataset

WRITES

Program → Dataset

UPDATES

Program → Table

BELONGS_TO

Program → Business Capability

IMPLEMENTS

Program → Business Rule

CONTAINS

Business Capability → Program

Example Graph

PAYMENTJOB ↓ EXECUTES

PAYPOST ↓ CALLS

PAYUPD ↓ UPDATES

CARD_ACCOUNT

Core Use Cases

Discovery

Questions:

- What programs exist?
 - What jobs exist?
 - What tables exist?
-

Dependency Analysis

Questions:

- Who calls this program?
 - Which jobs execute this program?
-

Data Lineage

Questions:

- Which programs update this table?
 - Which jobs load this dataset?
-

Root Cause Analysis

Questions:

- Why did the batch fail?
 - What downstream systems are impacted?
-

Impact Analysis

Questions:

- What breaks if this table changes?
- What breaks if this copybook changes?

Knowledge Transfer

Questions:

- Explain this program.
- Explain this job flow.
- Explain this business process.

Modernization Planning

Questions:

- What services can be extracted?
- What domains exist?
- What modernization path should be chosen?

Modernization Strategy Framework

Phase 1 - Discover

Objective:

Understand what exists.

Deliverables:

- Repository Inventory
- Program Catalog
- Job Catalog

Phase 2 - Understand

Objective:

Understand relationships.

Deliverables:

- Call Graph
 - Data Lineage
 - Dependency Graph
-

Phase 3 - Explain

Objective:

Understand business behavior.

Deliverables:

- Business Rules
 - Capability Maps
 - Functional Documentation
-

Phase 4 - Reason

Objective:

Answer enterprise questions.

Deliverables:

- Impact Analysis
 - Flow Analysis
 - Root Discovery
-

Phase 5 - Modernize

Objective:

Prepare transformation.

Deliverables:

- Service Candidates
 - Domain Boundaries
 - API Candidates
 - Event Candidates
-

Phase 6 - Transform

Objective:

Execute modernization.

Deliverables:

- Service Specifications
 - Migration Plans
 - Test Recommendations
 - Transformation Roadmaps
-

Focus Areas For Modernization

Business Rule Discovery

Extract hidden business logic.

Capability Mapping

Map systems to business functions.

Dependency Elimination

Identify tight coupling.

Service Identification

Discover service boundaries.

API Discovery

Identify reusable capabilities.

Data Lineage

Understand data movement.

Event Identification

Discover event-driven opportunities.

Test Intelligence

Identify regression impacts automatically.

Knowledge Preservation

Capture institutional knowledge permanently.

Success Metrics

Reduce:

- SME dependency
- Impact analysis time
- Onboarding time
- Modernization risk
- Documentation gaps

Increase:

- Developer productivity
 - System visibility
 - Change confidence
 - Modernization readiness
 - Knowledge retention
-

Long-Term Vision

A user should be able to ask:

"How does payment posting work?"

and receive:

- Business capability
- Jobs involved
- Programs involved
- Tables involved
- Dependencies
- Business rules
- Impact analysis
- Modernization recommendations

within seconds, without requiring SME intervention.

MIP becomes the enterprise intelligence layer for legacy systems and the foundation for safe, scalable modernization.